

1) Recap

2) Feed Forward Neural Networks (FFNN)

2.1) Recap On Perceptron

The perceptron is a **binary classification algorithm**, that is:

$$f: \mathbb{R}^d \rightarrow \{-1, +1\}, \quad x \rightarrow t$$

It divides the space into two regions divided by a hyperplane. These regions are called **halfspaces**:

$$H = \{x \rightarrow \text{sign}(w^T x + b) : w \in \mathbb{R}^d, b \in \mathbb{R}\}$$

The decision function f is the sign: $y = f(x) = \text{sign}(w^T x + b)$

Hyperplane

The hyperplane is the points that satisfy $w^T x = K$. The hyperplane has direction given by w and the distance from the origin is $w^T x = \|w\| \|x\| \cos(\theta)$.

Now add the bias b . The hyperplane is the set of points that satisfy

$$w^T x + b = 0$$

and we define v as the vector from the origin to the closest point of the hyperplane (perpendicular)

$$v = d \frac{w}{\|w\|} = d \cdot u_w$$

The distance can be found by substituting $x = v$:

$$w^T x + b \stackrel{x=v}{=} w^T d \frac{w}{\|w\|} + b \stackrel{w^T w = \|w\|^2}{=} d \|w\| + b = 0 \rightarrow d = -\frac{b}{\|w\|}$$

From here we get that: **w gives the orientation and b the distance from origin**

for convenience the bias is added as a weight:

$$w_d = \begin{bmatrix} w_1 \\ \dots \\ w_d \end{bmatrix}, x_d = \begin{bmatrix} x_1 \\ \dots \\ x_d \end{bmatrix} \rightarrow w_{d+1} = \begin{bmatrix} w_1 \\ \dots \\ w_d \\ b \end{bmatrix}, x_{d+1} = \begin{bmatrix} x_1 \\ \dots \\ x_d \\ 1 \end{bmatrix}$$

so that $y = \text{sign}(w_{d+1}^T x_{d+1}) = \text{sign}(w_d^T x_d + b)$.

Training

Given a dataset of labeled points:

$$\mathcal{D} = \{(x_1, t_1), \dots, (x_N, t_N)\} \quad t_n \in \{-1, +1\}$$

start with $w^{(0)} = \vec{0}$

A correct classification is when $(w^{(i)})^T x_n t_n > 0$.

We must minimize the loss function:

$$\min \mathcal{L}(w, \mathcal{D}) = \min \sum_{n \in \mathcal{D}} l(w, x_n) = \min \sum_{n \in \mathcal{D}} \max(0, -w^T x_n t_n)$$

Weights are updated when a sample is incorrectly classified:

$$w^{(i+1)} = w^{(i)} + x_n t_n$$

At each iteration we are always improving the loss function

Proof:

$$(w^{(i+1)})^T x_n t_n = (w^{(i)} + x_n t_n)^T x_n t_n = (w^{(i)})^T x_n t_n + \|x_n\|^2 > (w^{(i)})^T x_n t_n$$

□

Limitations

This only works for linearly separable data.

A possible solution is feature mapping:

$$\phi(x), \phi : \mathbb{R}^d \rightarrow \mathbb{R}^m$$

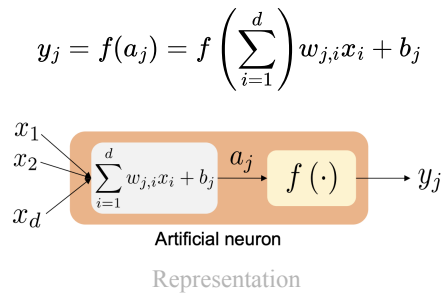
but a good ϕ is hard to find. **Neural networks learn the feature mapping**

There are infinitely many feature mappings, we base them on two insights:

1. ϕ is built as a composition of multiple simpler functions ($\phi = f \circ g \circ h \circ \dots$)
2. ϕ has a multi-layered structure with increasing abstraction

2.2) FFNN

NN are able to learn the feature mapping. Each smaller function (neuron) is based on the perceptron. Instead of a sign function a differentiable activation function is used.



2.3) Single Layer FFNN

A layer of a FFNN is a **stack of multiple neurons**.

The input vector is fed to a hidden layer (neurons) that have:

-An affine transformation

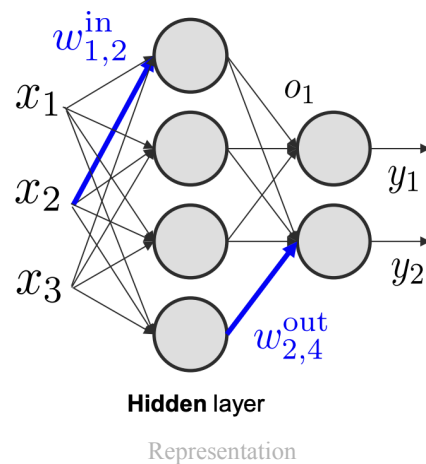
$$a = W^{in} z + b^{in}$$

-A non-linear activation function

$$o = f(a)$$

The output is therefore:

$$y = g(W^{out} o + b^{out})$$



Theorem 1 (Universal Approximation Capability).

Single-layer FFNN can approximate any function with arbitrary accuracy given a suitable activation function

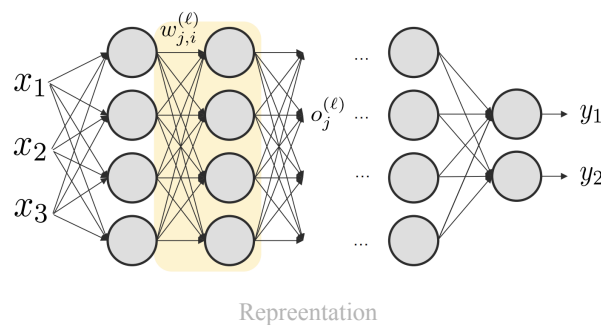
The number of hidden neurons required is exponential in the dimension of the input space d . This means single-layer FFNNs are of limited use in solving practical problems. Multiple layers can be used to solve this problem.

2.4) Multi-Layered FFNN

Notation

Start with some notation:

- $w_{i,j}^{(l)}$: weights that connects neuron i of layer $l - 1$ with neuron j of layer l
- $W^{(l)}, b^{(l)}$: parameters of layer l
- $a_j^{(l)}, o_j^{(l)}$: activation and output of neuron j of layer l
- $\mathcal{W}$: set of FFNN parameters (weights and biases)
- $\mathcal{L}(\mathcal{W}, x)$: loss function of FFNN on input sample x
- $\mathcal{D} = \{(x_1, t_1), \dots, (x_N, t_N)\}$: labeled training data



recall: $w_{j,i}$ means direction $i \rightarrow j$.

Input-Output Relation: Forward Pass

Given an input x , the output is obtained with the forward pass:

- feed x to the first layer: $a^{(1)} = W^{(1)}x + b^{(1)}$, $o^{(1)} = f(a^{(1)})$
- Use $o^{(1)}$ as input for the second layer
- Repeat for all layers
- Get output vector as: $a^{(L)} = W^{(L)}o^{(L-1)} + b^{(L)}$, $y = g(a^{(L)})$

$a^{(l)}, o^{(l)} \forall l \in \{1, \dots, L\}$ are stored for the back propagation.

Loss Function

The loss function represents the optimization objective of the NN training. It should reflect the learning task.

Typically:

- Negative log-likelihood (NLL) of parameters
- Assume **i.i.d** training samples $\rightarrow$ decompose NLL as sum of terms

$$\mathcal{L}(\mathcal{D}, \mathcal{W}) = -\log p(\{t_n\}_{n=1}^N | \{x_n\}_{n=1}^N; \mathcal{W}) = -\sum_{n=1}^N \log p(t_n | x_n; \mathcal{W})$$

Regression (Continuous Labels and Output)

OUTPUT FUNCTION

for now g is the identity function, therefore $y_n = a_n^{(L)}$
It allows to span the whole target space $\mathbb{R}^n$

ERROR FUNCTION: MEAN SQUARED ERROR (MSE)

Suppose that label is given by the output + some gaussian noise: $t_n = y_n + \text{noise}$
 Then the **per sample likelihood** can be written as a gaussian:

$$\underbrace{p(t_n|x_n; \mathcal{W})}_{\text{objective}} = \mathcal{N}(t_n; y_n, \Sigma)$$

where we suppose y_n is the mean and Σ the covariance.

By setting $\Sigma = I$ so to focus on predicting the mean we can find the following loss function:

$$\begin{aligned} -\log(\mathcal{N}(t_n; y_n, \Sigma)) &= \frac{1}{2} [\log \det \Sigma + d \log 2\pi + (y_n - t_n)^T \Sigma^{-1} (y_n - t_n)] \\ &\stackrel{\Sigma=I}{=} \frac{1}{2} [d \log 2\pi + (y_n - t_n)^T (y_n - t_n)] \\ &\quad \downarrow \\ \mathcal{L}(\mathcal{D}, \mathcal{W}) &\propto \frac{1}{2} \sum_{n=1}^N \|y_n - t_n\|^2 \end{aligned}$$

Classification (Discrete Labels, Continuous Output)

Since this is a classification problem, consider the binary classification case:

- 1 output neuron with $t_n \in \{0, 1\}$
- The two classes are called $\mathcal{C}_0, \mathcal{C}_1$

OUTPUT FUNCTION

Usually the sigmoid function is used:

$$y_n = g(a_n^{(L)}) = \sigma(a_n^{(L)}) = \frac{1}{1 + \exp \left\{ -a_n^{(L)} \right\}}$$

This maps $\mathbb{R} \rightarrow [0, 1]$

This can be interpreted as the probability of the output being 1 given the input: $p(\mathcal{C}_1|x_n)$

Or alternatively the *complementary class*: $1 - y_n = p(\mathcal{C}_0|x_n)$.

ERROR FUNCTION: CROSS-ENTROPY

Recalling the complementary class, the likelihood can be expressed as:

$$p(t_n|x_n; \mathcal{W}) = p(\mathcal{C}_1|x_n)^{t_n} p(\mathcal{C}_0|x_n)^{1-t_n}$$

and the NLL becomes:

$$\begin{aligned} -\log p(t_n|x_n; \mathcal{W}) &= -t_n \log p(\mathcal{C}_1|x_n) - (1 - t_n) \log p(\mathcal{C}_0|x_n) \\ &= -t_n \log y_n - (1 - t_n) \log(1 - y_n) \\ \mathcal{L}(\mathcal{D}, \mathcal{W}) &= - \sum_{n=1}^N \log p(t_n|x_n; \mathcal{W}) \\ &= - \sum_{n=1}^N t_n \log(y_n) + (1 - t_n) \log(1 - y_n) \end{aligned}$$

GENERALIZATION TO MULTI-CLASS

This can be generalized to multi-class classification problems.

- Let there be Q labels
- Labels are expressed in one-of-Q representation (one-hot encoding)
- Use Q output neurons, each one dedicated to one of the labels

if x_n belongs to a class q , then set $t_{n,q} = 1, t_{n,i} = 0 \forall i \neq q$.

this can be seen as $y_{n,q} = p(t_{n,q} = 1|x_n)$

The error function is the **categorical cross entropy**

$$\mathcal{L}(\mathcal{D}, \mathcal{W}) = - \sum_{n \in N} \sum_{q \in Q} t_{n,q} \log(y_{n,q})$$

SOFTMAX OUTPUT

In multi-class, the **output should be a discrete probability distribution over the Q classes**. Therefore:

- $y_{n,q} \in [0, 1], \forall q$
- $\sum_{q \in Q} y_{n,q} = 1$

The most common choice is the **softmax function**

$$y_{n,q} = \frac{\exp \{a_q^{(L)}\}}{\sum_{i \in Q} \exp \{a_i^{(L)}\}}$$

2.5 FFNN Training (GD & BP)

The weight are updated via the **gradient descent** rule

$$(w_{j,i}^{(l)})^{(n+1)} = (w_{j,i}^{(l)})^{(n)} - \eta \nabla_{w_{j,i}^{(l)}} L(W, x)$$

This is calculated via the **backpropagation algorithm**:

1. Execute a forward pass to obtain and store the activations and outputs at each layer
2. For the specific loss and output function, compute

$$\delta_m^{(L)} = \frac{\partial L}{\partial y_m} g'(a_m^{(L)})$$

3. Via the backward pass, compute

$$\delta_j^{(l)} = f'(a_j^{(l)}) \sum_k \delta_k^{(l+1)} w_{k,j}^{(l+1)}$$

4. Finally obtain the gradients for all weights/biases:

$$\frac{\partial L}{\partial w_{j,i}^{(l)}} = \delta_j^{(l)} o_i^{(l-1)} \quad \frac{\partial L}{\partial b_j^{(l)}} = \delta_j^{(l)}$$

Mathematical Derivation

The aim of backpropagation is to compute the gradient of the loss function with respect to all weights and biases of the network., that is

$$\frac{\partial L(W, x)}{\partial w_{j,i}^{(l)}}, \frac{\partial L(W, x)}{\partial b_j^{(l)}} \quad \forall i, j, l$$

Via the chain rule of derivatives this problem can be recursive and efficient

FORWARD PASS

The forward pass consists in computing the activations and outputs at each layer of the network given an input vector:

We will store all of the following quantities:

$$o^{(l)}, a^{(l)} \quad \forall l, y$$

CHAIN RULE

Consider the weight $w_{j,i}^{(l)}$ (going from output i at layer $l - 1$ to activation j at layer l). The activation will be influenced by the weight (and bias) since

$$a_j^{(l)} = \sum_n w_{j,n}^{(l)} o_n^{(l-1)} + b_j^{(l)}$$

and the two derivatives become:

$$\begin{aligned} \frac{\partial L}{\partial w_{j,i}} &= \frac{\partial L}{\partial a_j^{(l)}} \frac{\partial a_j^{(l)}}{\partial w_{j,i}} = \frac{\partial L}{\partial a_j^{(l)}} o_i^{(l-1)} \\ \frac{\partial L}{\partial b_j^{(l)}} &= \frac{\partial L}{\partial a_j^{(l)}} \frac{\partial a_j^{(l)}}{\partial b_j^{(l)}} = \frac{\partial L}{\partial a_j^{(l)}} \end{aligned}$$

From here define the **error message** as

$$\delta_j^{(l)} \triangleq \frac{\partial L}{\partial a_j^{(l)}}$$

which only depends on layers $> l$.

This can be recursively computed as:

$$\delta_j^{(l)} = \sum_k \underbrace{\frac{\partial L}{\partial a_k^{(l+1)}}}_{\delta_k^{(l+1)}} \frac{\partial a_k^{(l+1)}}{\partial a_j^{(l)}}$$

Then the first term is the error message of the $l + 1$ layer while the second term can be computed as

$$\frac{\partial a_k^{(l+1)}}{\partial a_j^{(l)}} = \frac{\partial a_k^{(l+1)}}{\partial o_j^{(l)}} \frac{\partial o_j^{(l)}}{\partial a_j^{(l)}} = w_{k,j}^{(l+1)} f'(a_j^{(l)})$$

(since $o_j = f(a_j)$ and $a_k^{(l+1)} = \sum_m w_{k,m}^{(l+1)} o_m^{(l)} + b_k^{(l+1)}$)

Then finally:

$$\delta_j^{(l)} = f'(a_j^{(l)}) \sum_k \delta_k^{(l+1)} w_{k,j}^{(l+1)}$$

2.6) Activation Functions

A plethora of functions can be used as activation functions.

Sigmoid Activation Function

$$\begin{aligned} f(a_j^{(l)}) &= \sigma(a_j^{(l)}) = \frac{1}{1 + \exp\{-a_j^{(l)}\}} \\ f'(a_j^{(l)}) &= \sigma(a_j^{(l)}) \cdot (1 - \sigma(a_j^{(l)})) \end{aligned}$$

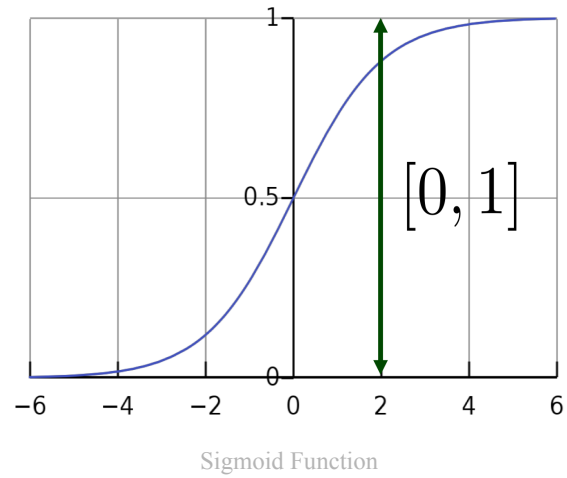
This is a **soft and differentiable function** that maps $\mathbb{R} \rightarrow [0, 1]$.

However the **derivative is contractive**, that is it gets smaller when the input gets bigger in magnitude. This makes the gradient vanish! The sigmoid works for shallow FFNN ($L < 8$).

Another drawback is the **bias drift** since the output is not zero centered, that is, there is an additive bias:

$$\mathbb{E}[a] = 0 \rightarrow \mathbb{E}[\sigma] \neq 0$$

This slows down the learning process.

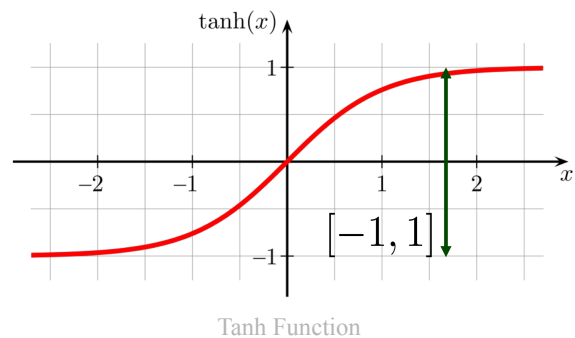


Hyperbolic Tangent (tanh) Activation Function

$$f(a_j^{(a)}) = \tanh(a_j^{(l)}) = \frac{e^{a_j^{(a)}} - e^{-a_j^{(a)}}}{e^{a_j^{(a)}} + e^{-a_j^{(a)}}}$$

$$f'(a_j^{(l)}) = 1 - \tanh^2(a_j^{(a)})$$

This is a scaled version of the sigmoid function.
This maps $\mathbb{R} \rightarrow [-1, 1]$ and is differentiable. The **derivative is still contractive** but it **solves the bias drift problem**



Linear Activation Functions

A composition of linear functions is linear and can be expressed as a matrix product.
 a_j is a linear function and therefore also $f(a_j)$ is.

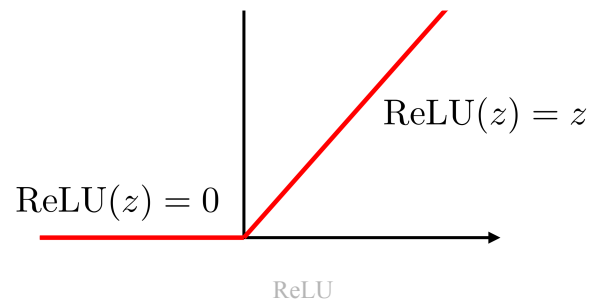
RECTIFIED LINEAR UNIT (ReLU) ACTIVATION FUNCTION

$$ReLU(a_j^{(l)}) = \max(0, a_j^{(l)})$$

$$ReLU(a_j^{(l)}) = \begin{cases} 0, & z < 0 \\ 1, & z > 0 \end{cases}$$

Simplicity of linear function but non-linear. This speeds up the learning process and returns a sparse output

For $z > 0$ the derivative is 1, but ReLU can die out if the bias of a layer becomes too negative:



$$a_j^{(l)} = \sum \dots \underbrace{-10^{10}}_b < 0 \rightarrow f'(a_j^{(l)}) = 0$$

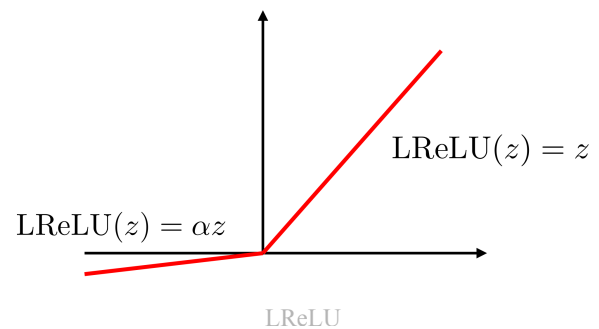
Moreover the output is not zero centered.

LEAKY ReLU (LReLU) ACTIVATION FUNCTION

$$LReLU(z) = \begin{cases} \alpha z, & z < 0 \\ z, & z \geq 0 \end{cases}$$

$$LReLU'(z) = \begin{cases} \alpha, & z < 0 \\ 1, & z > 0 \end{cases}$$

This mitigates the dying ReLU problem but gives less sparse output. Moreover the magnitude increases for large negative values and could amplify the noise.

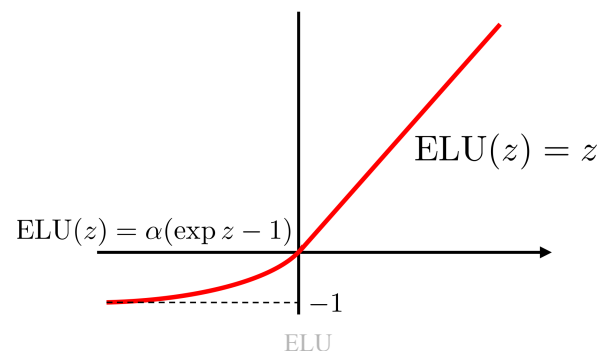


EXPONENTIAL LINEAR UNIT (ELU) ACTIVATION FUNCTION

$$ELU(z) = \begin{cases} \alpha(\exp\{z\} - 1), & z < 0 \\ z, & z \geq 0 \end{cases}$$

$$ELU'(z) = \begin{cases} \alpha \exp\{z\}, & z < 0 \\ 1, & z > 0 \end{cases}$$

This saturates for large negative values and is more robust to noise, however it is more complex to compute and can result in a slower learning process.



2.7) Parameters

Initialization

A bad initialization can result in a slow learning process, but an analytical solution is not available: therefore **initialization strategies are random and heuristic**

These are the common sense rules:

- Weight should be initialized randomly to ensure **diversity**
- No large values to avoid **instability and numerical issues**
- Common choice is zero mean gaussian distribution with small variance
- Another common choice is the uniform distribution on the layer input size

GLOROT INITIALIZATION

Default initialization for most DL frameworks:

$$w_{j,i}^{(0)} \sim u \left(-\sqrt{\frac{6}{d+n}}, \sqrt{\frac{6}{d+n}} \right)$$

Where d, n are the input and output dimensions of the network

Biases are initialized to zero.

Hyperparameters

We call hyper-parameters of a NN all the additional parameters that are **not learned**

LEARNING RATE

We call an **epoch** one entire run of the training phase.

TODO

2.8) Limitations

Since each layer is 1D we have:

- **large number of parameters** to learn.
- **loss of structural information**

Example (Size).

A 256px square RGB image has size $256 \times 256 \times 3 \rightarrow 196608 \times 1$ vector

Let each layer be made of 50 neurons, one single layer brings to $196608 \times 50 = 9.8M$ parameters and occupies roughly $9.8M \times 32bits = 315Mb$

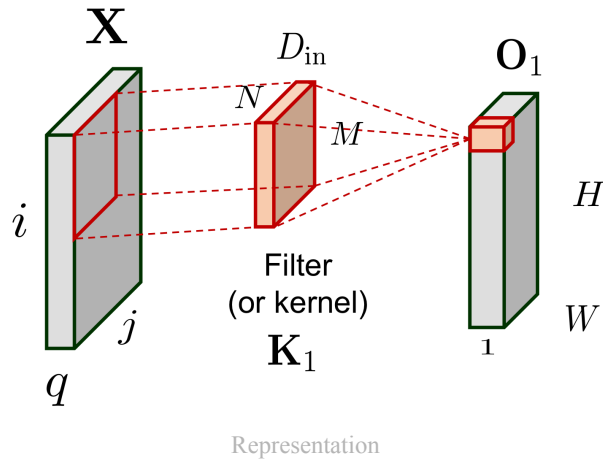
3) Convolutional Neural Network (CNN)

3.1) Intro to CNNs

CNNs support input of up to 3 dimensions i, j, q (length, height, depth).

They replace Matrix Multiplication with Convolutional (cross-correlation) Operations:

$$o_{l,i,j} = f \left(\underbrace{\sum_{q=1}^{D_{in}} (K_{l,q} * X_q)(l, i, j) + b}_{\text{activation map}} \right) \quad l = 1, \dots, D_{out}$$



1. $\sum$: sum over the input depth
2. $(*)()$: convolve filter with input at depth q
3. b : adds bias, f : activation function

In general at every layer we have D_{out} filters (hyperparameter) that is slid over the whole input to generate a new activation map. The collection of D_{out} feature maps is sent to the next layer.

Notice that each filter is a collection of weights that will be randomly initialized and learned during the training phase.

Notice that usually the convolution is defined as:

$$(X * D)(i, j) = \sum_m \sum_n X(i - m, j - n) K(m, n)$$

but CNNs use a different definition, that is the **cross-correlation** which corresponds to a 180° rotation of the filter and practically results changing the sign:

$$(X * D)(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

The rotation will be learned by the network, so we don't need to rotate it manually. Moreover cross-correlation is efficient to implement and is a **similarity** indicator

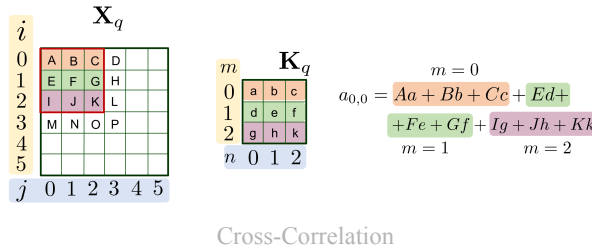
Definitions

- **Feature Map:** result of convolving one filter of the layer with the whole input and applying the activation function (element-wise)
- **Stride:** number of steps by which the filter is shifted
- **(Zero) Padding:** add zeros to the input so that the filter fits an integer number of times (symmetrically on all sides)

Now call Stride S , Padding P , Filter size F and input dimension d , then the output dimension is (one for each D_{out}):

$$O = \frac{d - F + 2P}{S} + 1$$

An easy way to see the cross-correlation computation:



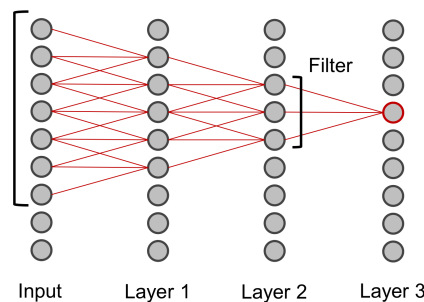
3.2) Strengths

Local Connectivity

Since the size of the filter is much smaller than the input we have some advantages:

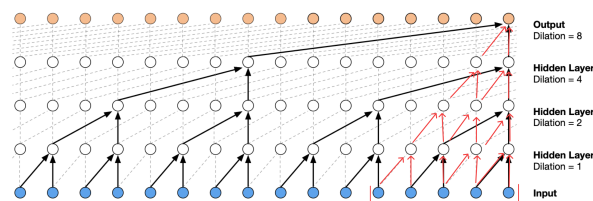
- Each neuron depends on a small subset of inputs of previous layers (filter size) (FFNN uses all neurons for next layer single neuron)
- Patterns are learned in a hierarchical way, each layer learns patterns of the previous layer, stacking them creates global patterns

This can be seen by analyzing the **receptive field**: Number of elements in the input that influence a given neuron at a given layer.



Receptive Field Example

The receptive field can be increased by **dilating** the convolution. Instead of using subsequent features, at each layer the filter is dilated so that the receptive field is maximized.



Non Dilated In Red

The **size of receptive field increases with its depth** → neurons closer to input depend on local features, deeper neuron on global ones (generalization of local features)

This shows that CNNs work well on structured data

Weight Sharing

Since weights are shared in the filter, the number of parameters is drastically reduced

Example.

Recall the $256 \times 256 \times 3$ RGB image.

- CNN with 3×3 filter has 9 weights per depth 27 total weights per output feature map, we want 3 output maps so 81 weights
- FFNN in one layer would have $(256 \times 256 \times 3) \times (256 \times 256 \times 3) = 38M$ weights

this also brings:

- higher efficiency in encoding information (spatial structure)
- less overfitting
- more layers, deeper features

Translation Equivariance

First define equivariance:

Definition 2 (Equivariance).

Given two functions ϕ, ξ and an input x , they are equivariant if:

$$\phi(\xi(x)) = \xi(\phi(x))$$

Let the functions be a convolution and a translation, then the equivariance holds. In neural networks terms:

If a pattern in the input is shifted, the learned features are shifted but unchanged

Quick proof:

Translate the convolution

$$(f * g)(x) \xrightarrow{\text{translate}} (f * g)(x - z) = \int f(\tau)g(x - z - \tau)d\tau$$

Now translate f and then convolve

$$f(x - z) * g(x) = \int f(\tau - z)g(x - \tau)d\tau \stackrel{u=\tau-z}{=} \int f(u)g(x - z - u)du = (f * g)(x - z)$$

□

Remark.

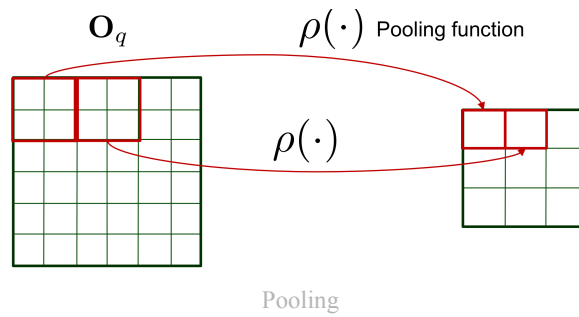
Equivariant to translations doesn't mean equivariant to rotations!

Pooling Layers

It can be useful to reduce the dimension of the tensors between layers.

This can be achieved by:

- Using $S > 1$
- Pooling layers



This adds **invariance** to small translations. Useful for classification (object present, not where it is present)

Usually the pooling function is a mean or max function

In BP (see later) the pooling affects the error message in two possible ways:

- Max pooling: error message is assigned to the max value of the patch
- Avg pooling: all inputs receive error message but it is divided by the surface of the pooling patch

3.3) Training (SGD & BP)

Recall some notation:

- Layers: $l = 1, \dots, L$
- Input feature map: Size $H \times W$ indices i, j
- Filter: Size $M \times N$ indices m, n
- Weight between layers $l - 1, l$: $w_{m,n}^l$, bias b
- Activation (conv of prev. output and filter)

$$a_{i,j}^l = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} o_{i+m,j+n}^{l-1} w_{m,n}^l + b^l$$

In CNNs the last layer is usually a FFNN (easier to have 1D for regression and classification). CNN learns features, FFNN classifies/regress based on features.

The following steps are used:

1. Apply forward pass to compute all outputs and activations $a_{i,j}^l, o_{i,j}^l$
2. Last layer is FFNN, so compute error message

$$\delta^L = \nabla_y L \odot f'(\mathbf{a}^L) = \begin{bmatrix} \delta_1^L \\ \vdots \\ \delta_M^L \end{bmatrix}, \delta_i^l = \frac{\partial L}{\partial y_i} f'(a_i^L)$$

3. Recursively compute error signal matrices

$$\delta^l = f'(a_{i,j}^l) \delta^{l+1} * \text{rot}(180^\circ)(K^{l+1})$$

4. Compute gradient using error signal matrices

$$\frac{\partial L(W, X)}{\partial w_{m,n}^l} = (\delta^l * \mathbf{o}^{l-1})(m, n) \quad \frac{\partial L(W, X)}{\partial b^l} = \sum_{i=0}^{H-M} \sum_{j=0}^{W-N} \delta_{i,j}^l$$

The learning rule is the SGD:

$$w_{m,n}^{(l+1)} = w_{m,n}^{(l)} - \eta \frac{\partial L(W, X)}{\partial w_{m,n}^{(l)}} \quad b^{(l+1)} = b^{(l)} - \eta \frac{\partial L(W, X)}{\partial b^{(l)}}$$

MATHEMATICAL DERIVATION

Back propagation analysis with $S = 1, P = 0, D_{in} = D_{out} = 1$ but can be generalized

The aim is to compute the gradient of the loss function wrt the weights:

$$\frac{\partial L(W, X)}{\partial w_{m,n}^l} \stackrel{\text{chain rule}}{=} \sum_{i=0}^{H-M} \sum_{j=0}^{W-N} \underbrace{\frac{\partial L(W, X)}{\partial a_{i,j}^l}}_{\delta_{i,j}^l} \underbrace{\frac{\partial a_{i,j}^l}{\partial w_{m,n}^l}}_{o_{i+m,j+n}^l} = (\delta^l * \mathbf{o}^{l-1})(m, n)$$

where $\delta_{i,j}^l$ is the error signal and $o_{i+m,j+n}^l$ is obtained like in FFNN where the derivative is 0 besides for $i, j = m, n$ (in the activation this results in the output at $i + m, j + n$).

The gradient is therefore just the convolution of the error signal and the output.

How does the loss function behave wrt the activations? They are not fully connected, so how is the error signal computed?

In general the output $o_{i,j}^l$ is used in the calculation of the activations $a_{i-n,j-m}^{l+1}$ where $n, m \in \{0, 1\}$ and the indices $i - n \in \{0, \dots, H - N + 1\}, j - m \in \{0, \dots, W - M + 1\}$.

Knowing this the error signal can be computed as

$$\delta_{i,j}^l = \frac{\partial L(W, X)}{\partial a_{i,j}^l} = \sum_m \sum_n \underbrace{\frac{\partial L(W, X)}{\partial a_{i-n,j-m}^{l+1}}}_{\delta_{i-m,j-n}^{l+1}} \frac{\partial a_{i-n,j-m}^{l+1}}{\partial a_{i,j}^l}$$

which brings to the calculation of

$$\frac{\partial a_{i-n,j-m}^{l+1}}{\partial a_{i,j}^l} = \frac{\partial}{\partial a_{i,j}^l} \left(\sum_{m'} \sum_{n'} w_{m',n'}^{l+1} f(a_{i-m+m',j-n+n'}^l) + b^{l+1} \right) \stackrel{m'=m, n'=n}{=} w_{m,n}^{l+1} f'(a_{i,j}^l)$$

so the error message is

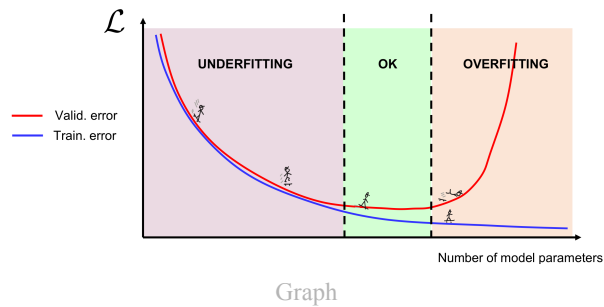
$$\delta_{i,j}^l = \sum_m \sum_n \delta_{i-m,j-n}^{l+1} w_{m,n}^{l+1} f'(a_{i,j}^l) \longrightarrow \delta^l = f'(a_{i,j}^l) \delta^{l+1} * \text{rot}(180^\circ)(K^{l+1})$$

todo update bias.

4) Overfitting and Regularization in Neural Networks

NN can easily have $> 100M$ parameters and therefore overfitting is a common problem.

Consider the validation vs training error:



The **train-validation** curves are essential in understanding the status of the model. When the training and validation error start to diverge, the model is overfitting

Definition (Early Stopping).

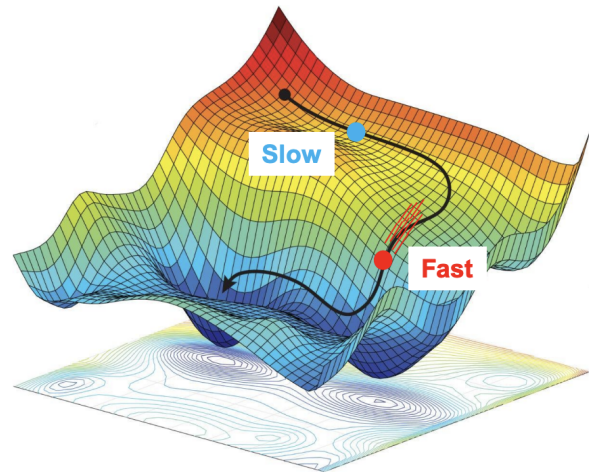
Early stopping is a common technique to prevent overfitting by stopping training when the validation error starts to diverge from the training error.

5) Optimization Methods for Neural Networks

Momentum

Imagine the loss function as a landscape. Imagine a ball rolling on the landscape. The slope makes it descent but constant slope in the same direction gives it momentum. More momentum means more force necessary to divert the ball → This region gives robustness to momentary changes in the update direction.

How do we give SGD this property?



Example

SMOOTHING

Find the **cumulative average**

$$h^{(k)} = \sum_{i=1}^k \frac{w^{(i)}}{k} \quad \text{Recursively} \quad \underbrace{\frac{k-1}{k}}_{\alpha} h^{(k-1)} + \underbrace{\frac{1}{k}}_{\beta} w^{(k)}$$

with $\alpha + \beta = 1$

The recursive form is used to compute average without needing to save all of the weights at every time k

Samples are all of equal weight

We can also use the **exponential average**

$$h^{(k)} = \beta h^{(k-1)} + (1 - \beta) w^{(k)}$$

with $\beta \in [0, 1]$ that controls the smoothness of the trajectory.

Samples are not of equal weight: more recent samples are more valuable

Return to the ball metaphor

- Position of ball: current parameters of vector $w^{(i)}$
- Velocity of ball $v^{(i)}$ is equal to momentum of unit mass ball
- Gravity force = gradient of the loss function

$$\frac{\eta^{(i)}}{K} \sum_{k=1}^K \nabla L(x_k, t_k; w^{(i)})$$

- Update rule:

$$v^{(i+1)} = \alpha v^{(i)} - \frac{\eta^{(i)}}{K} \sum_{k=1}^K \nabla L(x_k, t_k; w^{(i)})$$

$$w^{(i+1)} = w^{(i)} + v^{(i+1)}$$

Only the momentum is stored, not the gradients or weights.

Call $g^{(i)} = \frac{1}{K} \sum_{k=1}^K \nabla L(x_k, t_k; w^{(i)})$. Now we can see that: (TO FIX THIS FORMULA THERE IS AN ERROR)

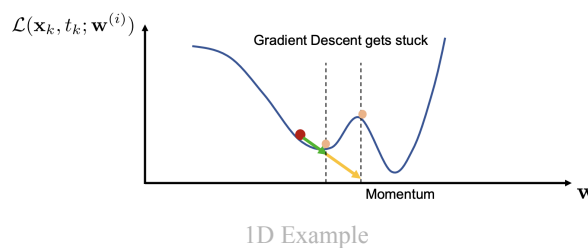
$$\begin{aligned} v^{(1)} &= -\eta^{(0)} g^{(0)} \\ v^{(2)} &= \alpha v^{(1)} - \eta^{(1)} g^{(1)} = \alpha(-\eta^{(0)} g^{(0)}) - \eta^{(1)} g^{(1)} \\ &\vdots \\ v^{(k)} &= \alpha^{k-1}(-\eta^{(0)} g^{(0)}) + \alpha^{k-2}() \dots + \alpha(-\eta^{(k-1)} g^{(k-1)}) - \eta^{(k)} g^{(k)} \end{aligned}$$

At first iteration we suppose the ball is at rest, moreover it shows that since $\alpha^n \xrightarrow{n \rightarrow \infty} 0$ it is clear that more recent weights are more important (similar to exponential smoothing).

Weight updates are less affected by quick changes in direction. Long-term trends in the gradient direction bring acceleration

For gradient based optimization it:

- Accelerates convergence, smooths out wandering behavior
- Makes it easier to escape from local minima



VARIATION: NESTEROV MOMENTUM

Look-ahead method: anticipate update to increase responsiveness

This means to consider the gradient in a forward position (possible overshoot scenario) so to have better response:

$$\nabla L(x_k, t_k; w^{(i)} + \alpha v^{(i)})$$

Adaptive Learning Rate Methods

NORMALIZED GRADIENT DESCENT

Near stationary points the gradient is close to zero, so the learning gets slower. To solve this we ignore the magnitude of the gradient by normalizing it:

$$w^{(i+1)} = w^{(i)} - \eta \frac{g^{(i)}}{|g^{(i)}|}$$

So the magnitude of the gradient is ignored

$$|w^{(i+1)} - w^{(i)}| = \left| -\eta \frac{g^{(i)}}{|g^{(i)}|} \right| = \eta$$

In practice we use

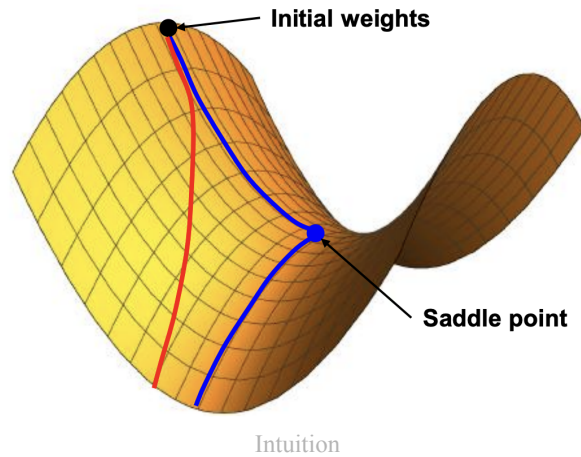
$$w^{(i+1)} = w^{(i)} - \eta \frac{g^{(i)}}{|g^{(i)}| + \epsilon}$$

with ϵ small (10^{-8})

AdaGRAD

New idea: component-wise adaptive learning rate (LR)

In fact using the same LR for all components is not recommended as some weights require slower/faster updates than others



Recall the gradient on a mini-batch at iteration i $g^{(i)}$
 Maintain a running sum of squared gradients **for m-th weight**

$$\alpha_m^{(i)} = \sum_{j=1}^i (g_m^{(j)})^2 = (g_m^{(i)})^2 + \alpha_m^{(i-1)}$$

And now compute the weight specific LR

$$\eta_m^{(i)} = \frac{\eta^{(0)}}{\sqrt{\alpha_m^{(i)} + \epsilon}}$$

and thus for the m-th weight:

$$w_m^{(i+1)} = w_m^{(i)} + \eta_m^{(i)} g_m^{(i)}$$

However the sum only grows so the learning rate decreases and might prematurely stop. Use exponentially decaying moving average:

$$r_m^{(i)} = \beta r_m^{(i-1)} + (1 - \beta)(g_m^{(i)})^2 = \sum_{k=1}^i \beta^{i-k} (1 - \beta)(g_m^{(k)})^2$$

this is an IIR filter

The exponents (from i to 1) decay exponentially fast: finite memory approximation by neglecting the terms that $\beta^j < T \rightarrow j > \log T / \log \beta$ are after the j -th iteration in the past

ADAPTIVE MOMENT ESTIMATION (ADAM)

Also keep track of the second moment:

$$\begin{aligned} \text{1st Gradient: } \xi_m^{(i)} &= \beta_1 \xi_m^{(i-1)} + (1 - \beta_1) g_m^{(i)} \\ \text{2nd Gradient: } \psi_m^{(i)} &= \beta_2 \psi_m^{(i-1)} + (1 - \beta_2) (g_m^{(i)})^2 \end{aligned}$$

But this is **not biased** since we set $g_m^{(0)} = 0$.

The bias correction just resorts to

$$\hat{\xi}_m^{(i)} = \frac{\xi_m^{(i)}}{1 - \beta_1^i} \quad \hat{\psi}_m^{(i)} = \frac{\psi_m^{(i)}}{1 - \beta_2^i}$$

Proof:
 todo

□